Req01:

```
DELIMITER //
DROP PROCEDURE IF EXISTS req01_update_staff_experience //

CREATE PROCEDURE req01_update_staff_experience()
BEGIN
  UPDATE staff
  SET years_experience = TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE())
  WHERE years_experience IS NULL
    OR years_experience <> TIMESTAMPDIFF(YEAR, hire_date, CURRENT_DATE());
END //
DELIMITER ;
-- To call this procedure:
-- CALL req01_update_staff_experience();
```

Req02:

```
DELIMITER //
DROP EVENT IF EXISTS req02_update_staff_experience_monthly //

CREATE EVENT req02_update_staff_experience_monthly
        ON SCHEDULE
                EVERY 1 MONTH
                STARTS (TIMESTAMP(DATE_FORMAT(CURRENT_DATE, '%Y-%m-01 08:00:00')))
        DO
                CALL req01_update_staff_experience();
//
DELIMITER ;
```

Req03: (using dbeaver it would not let me use the delimiter, so I just used the ordinary delimiter.

```
DELIMITER //
DROP TRIGGER IF EXISTS req03_auto_assign_bartender;

CREATE TRIGGER req03_auto_assign_bartender
BEFORE INSERT ON bartender
FOR EACH ROW
BEGIN
  DECLARE chosen_bar INT;
  -- Only act if the new bartender has no bar assigned (FK is NULL)
  IF NEW.id_bar IS NULL THEN
    -- Find bar with minimum number of bartenders
    SELECT b.id_bar
    INTO chosen_bar
    FROM bar AS b
    LEFT JOIN bartender AS bt
      ON bt.id_bar = b.id_bar
    GROUP BY b.id_bar
    ORDER BY COUNT(bt.id_bartender) ASC, b.id_bar ASC
    LIMIT 1;
    -- assign bar to the new row
    SET NEW.id_bar = chosen_bar;
  END IF;
END //
DELIMITER ;
```

```
DELIMITER //
DROP TRIGGER IF EXISTS req03_auto_assign_securitymember;

CREATE TRIGGER req03_auto_assign_securitymember
BEFORE INSERT ON            `security`
FOR EACH ROW
BEGIN
  -- all DECLAREs go here, before any other statement
  DECLARE chosen_stage INT;
  IF NEW.id_stage IS NULL THEN
    -- Find a stage (in the same festival) with minimum number of security members
    SELECT st.id_stage
    INTO chosen_stage
    FROM stage AS st
    LEFT JOIN `security` AS sec
      ON  sec.festival_name    = st.festival_name
      AND sec.festival_edition = st.festival_edition
      AND sec.id_stage         = st.id_stage
    WHERE st.festival_name    = NEW.festival_name
      AND st.festival_edition = NEW.festival_edition
    GROUP BY st.id_stage
    ORDER BY COUNT(sec.id_security) ASC, st.id_stage ASC
    LIMIT 1;
    -- assign stage to the new row
    SET NEW.id_stage = chosen_stage;
  END IF;
END //
DELIMITER ;
```

Req04:

```
DELIMITER //
DROP PROCEDURE IF EXISTS req04_update_type_of_music //

CREATE PROCEDURE req04_update_type_of_music()
BEGIN
  DECLARE v_name VARCHAR(255);
  DECLARE v_country VARCHAR(255);
  DECLARE done INT DEFAULT 0;
  DECLARE cur CURSOR FOR
                SELECT b.name, b.country
                FROM band b
                LEFT JOIN song s
                        ON b.type_of_music = s.type_of_music
                GROUP BY b.name, b.country, b.type_of_music
                HAVING COUNT(s.title) = 0;

  DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

  OPEN cur;

  read_loop: LOOP
     FETCH cur INTO v_name, v_country;
     IF done = 1 THEN
        LEAVE read_loop;
     END IF;
     UPDATE band
     SET type_of_music  = (
         SELECT s.type_of_music FROM song s
                        WHERE s.written_by = v_name
                        GROUP BY s.type_of_music
                        ORDER BY COUNT(*) DESC, RAND()
                        LIMIT 1
         )
     WHERE name = v_name AND country = v_country;
     END LOOP;
     CLOSE cur;

END //
DELIMITER ;
-- To call this procedure:
-- CALL req04_update_type_of_music();
```

Req05:

```
DELIMITER //
DROP FUNCTION IF EXISTS req05_rounded_decimal;
CREATE FUNCTION req05_rounded_decimal(x FLOAT)
RETURNS FLOAT
DETERMINISTIC
BEGIN
	-- our approach here was to declare a variable
	-- that its purpose was to check if the input variable
	-- had 2 decimals or less so to return it without modification
	-- if not then it is rounded
	DECLARE x_times_100 FLOAT;
	SET x_times_100 = x * 100;
	IF x_times_100 = ROUND(x_times_100) THEN
		RETURN x;
	ELSE
		RETURN ROUND(x, 2);
	END IF;
END //
DELIMITER ;
```

Req06:

```
DELIMITER //
DROP FUNCTION IF EXISTS req06_existence_curr_code;
CREATE FUNCTION req06_existence_curr_code(x CHAR(3))
RETURNS BOOL
DETERMINISTIC
BEGIN
	DECLARE upper_code CHAR(3);
	DECLARE exists_flag BOOL;
	SET upper_code = UPPER(x);
	SELECT (upper_code = 'EUR') OR EXISTS (
		SELECT 1
		FROM information_schema.COLUMNS
		WHERE table_schema = DATABASE()
		AND table_name   = 'eurofxref_hist'
		AND column_name  = upper_code
	)
	INTO exists_flag;
	IF exists_flag THEN
		RETURN TRUE;
	ELSE
		RETURN FALSE;
```

```
        END IF;
END //
DELIMITER ;
```

Req07:

```
USE G61_challange2_music_festival;
DELIMITER //
DROP PROCEDURE IF EXISTS req07_convert_currency //
CREATE PROCEDURE req07_convert_currency(
  IN Origin_Cur VARCHAR(3),
  IN Dest_Cur VARCHAR(3),
  IN Convert_Date DATE,
  INOUT Amount_To_Convert DECIMAL(10,2),
  OUT Error_Msg VARCHAR(100)
)
req07_convert_currency: BEGIN

        DECLARE Origin_Conversion FLOAT;
        DECLARE Dest_Conversion FLOAT;
        DECLARE query VARCHAR(250);

        SET @Dest_Conversion_Rate = NULL;
  SET @Origin_Conversion_Rate = NULL;

        SET Origin_Cur = UPPER(Origin_Cur);
        SET Dest_Cur = UPPER(Dest_Cur);
        SET Error_Msg = NULL;

        IF Amount_To_Convert <= 0 THEN
                SET Error_Msg = "Amount to convert can not be lower than or equal to 0.";
                SET Amount_To_Convert = NULL;
                LEAVE req07_convert_currency;
        END IF;

        IF Convert_Date > CURRENT_DATE THEN
                SET Error_Msg = "Conversion date must be today or in the past (not a future
date).";
                SET Amount_To_Convert = NULL;
                LEAVE req07_convert_currency;
        END IF;

        IF Origin_Cur = Dest_Cur THEN
                SET Error_Msg = "Currencies involved must be different from each other (no
conversion to self).";
        SET Amount_To_Convert = NULL;
                LEAVE req07_convert_currency;
        END IF;

  IF Origin_Cur != 'EUR' AND Dest_Cur != 'EUR' THEN
```

```
                SET Error_Msg = "The currency conversion has to always involve EUR (from or
to).";
                SET Amount_To_Convert = NULL;
                LEAVE req07_convert_currency;
        END IF;

        IF req06_existence_curr_code(Origin_Cur) = FALSE OR
req06_existence_curr_code(Dest_Cur) = FALSE THEN
                SET Error_Msg = "One or both of the currency codes involved are not valid (not
EUR and not in table).";
                SET Amount_To_Convert = NULL;
                LEAVE req07_convert_currency;
        END IF;


        IF Origin_Cur = 'EUR' THEN
                SET Origin_Conversion = 1.0;
                SET query = CONCAT(
          'SELECT `', LOWER(Dest_Cur), '`',
          ' INTO @Dest_Conversion_Rate
          FROM eurofxref_hist
          WHERE date = ?'
                );

                PREPARE state FROM query;
                EXECUTE state USING Convert_Date;
                DEALLOCATE PREPARE state;

                SET Dest_Conversion = @Dest_Conversion_Rate;

        ELSE
                SET Dest_Conversion = 1.0;

                SET query = CONCAT(
          'SELECT `', LOWER(Origin_Cur), '`',
          ' INTO @Origin_Conversion_Rate
          FROM eurofxref_hist
          WHERE date = ?'
                );

                PREPARE state FROM query;
                EXECUTE state USING Convert_Date;
                DEALLOCATE PREPARE state;

                SET Origin_Conversion = @Origin_Conversion_Rate;
        END IF;

        IF Origin_Conversion IS NULL OR Origin_Conversion <= 0 OR Dest_Conversion IS
NULL OR Dest_Conversion <= 0 THEN
    SET Error_Msg = "Conversion rate does not exist or is non-positive for the given date.";
    SET Amount_To_Convert = NULL;
    LEAVE req07_convert_currency;
```

```
END IF;


        IF Dest_Cur = 'EUR' THEN
    SET Amount_To_Convert = Amount_To_Convert / Origin_Conversion;
        ELSE
    SET Amount_To_Convert = Amount_To_Convert * Dest_Conversion;
        END IF;

        SET Amount_To_Convert = (SELECT req05_rounded_decimal(Amount_To_Convert));

END//
DELIMITER ;
```

Call 1: 100 EUR → USD = 107.79

```
SET @amount_to_convert = 100;
SET @error_message = NULL;
CALL req07_convert_currency('eur', 'usd', '2024-05-10', @amount_to_convert, @error_message);
SELECT @amount_to_convert AS converted_amount, @error_message AS error_message;
```

1 | Statistics 1

@amount_to_convert AS | Enter a SQL expression to filter results (use Ctrl+Space)

123 converted_amount	A-Z error_message
107.79	[NULL]

Call 2: 100 USD → EUR = 92.77

```
SET @amount_to_convert = 100;
SET @error_message = NULL;
CALL req07_convert_currency('usd', 'eur', '2024-05-10', @amount_to_convert, @error_message);
SELECT @amount_to_convert AS converted_amount, @error_message AS error_message;
```

1 | Results 1 (2) | Statistics 1

@amount_to_convert AS | Enter a SQL expression to filter results (use Ctrl+Space)

123 converted_amount	A-Z error_message
92.77	[NULL]

Call 3: Amount <= 0

```
SET @amount_to_convert = 0;
SET @error_message = NULL;
CALL req07_convert_currency('usd', 'eur', '2024-05-10', @amount_to_convert, @error_message);
SELECT @amount_to_convert AS converted_amount, @error_message AS error_message;
```

1 | Results 1 (2) | Results 1 (3) | Statistics 1

@amount_to_convert AS | Enter a SQL expression to filter results (use Ctrl+Space)

123 converted_amount	A-Z error_message
[NULL]	Amount to convert can not be lower than or equal to 0.

Call 4: Future date

```
SET @amount_to_convert = 100;
SET @error_message = NULL;
CALL req07_convert_currency('usd', 'eur', '2026-05-10', @amount_to_convert, @error_message);
SELECT @amount_to_convert AS converted_amount, @error_message AS error_message;
```

Results 1 (2) | Results 1 (3) | Results 1 (4) | Statistics 1

@amount_to_convert AS | Enter a SQL expression to filter results (use Ctrl+Space)

123 converted_amount	A-Z error_message
[NULL]	Conversion date must be today or in the past (not a future date).

Call 5: No EUR

```
SET @amount_to_convert = 100;
SET @error_message = NULL;
CALL req07_convert_currency('usd', 'jpy', '2024-05-10', @amount_to_convert, @error_message);
SELECT @amount_to_convert AS converted_amount, @error_message AS error_message;
```

Results 1 (2) | Results 1 (3) | Results 1 (4) | Results 1 (5) | Statistics 1

@amount_to_convert AS | Enter a SQL expression to filter results (use Ctrl+Space)

123 converted_amount	A-Z error_message
[NULL]	The currency conversion has to always involve EUR (from or to).